Shelby Map Black Pct

Python source rendered from shelby_map_black_pct.py.

Approximate line count: 367

```
#!/usr/bin/env python3
"""
Shelby County: selected low-growth community and Black population share by tract.

High-growth comparator is identified using the same methodology as final_model2.ipynb:
  - Spatially-constrained Ward agglomerative clustering (N=6 communities)
  - Connectivity matrix built from tract adjacency (touching polygons)
  - Features: ACS economic indicators (income, poverty, unemployment, LFPR)
    as proxies for IGS Economy pillar
  - Best non-low-growth community = highest median income / lowest poverty

Run:    python shelby_map_black_pct.py
Output: data/processed/presentation_ready/shelby_map_black_pct.png
"""

import os
import warnings
import requests
import numpy as np
import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt
import matplotlib.lines as mlines
from matplotlib.colors import Normalize
from matplotlib.cm import ScalarMappable
from scipy.sparse import coo_matrix
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from pathlib import Path

warnings.filterwarnings("ignore")

# ── Paths ──────────────────────────────────────────────────────────────────
_dir         = Path(os.path.dirname(os.path.abspath(__file__)))
SHP_PATH     = _dir / "data/raw/tl_2024_state_tracts/tl_2024_47_tract/tl_2024_47_tract.shp"
LOW_GRW_PATH = _dir / "data/processed/presentation_ready/selected_low_growth_96_tracts.geojson"
OUT_PATH     = _dir / "data/processed/presentation_ready/shelby_map_black_pct.png"
CLUSTER_CACHE = _dir / "data/processed/shelby_full_tract_cluster_map.geojson"


# ─────────────────────────────────────────────────────────────────────────────
# ACS data fetch
# ─────────────────────────────────────────────────────────────────────────────

def fetch_acs(year: int = 2022) -> pd.DataFrame:
    """
    Fetch ACS 5-year estimates for all Shelby County tracts.
    Variables: race, income, poverty, unemployment.
```

```
    """
    print(f"  Fetching ACS {year} 5-year estimates...")
    vars_needed = ",".join([
        "B02001_001E",   # total population
        "B02001_003E",   # Black or African American alone
        "B19013_001E",   # median household income
        "B17001_001E",   # population for poverty determination
        "B17001_002E",   # below poverty level
        "B23025_003E",   # civilian labor force
        "B23025_005E",   # unemployed
    ])
    r = requests.get(
        f"https://api.census.gov/data/{year}/acs/acs5",
        params={"get": vars_needed, "for": "tract:*", "in": "state:47 county:157"},
        timeout=60,
    )
    r.raise_for_status()
    rows = r.json()
    df   = pd.DataFrame(rows[1:], columns=rows[0])

    df["geoid"]     = df["state"].str.zfill(2) + df["county"].str.zfill(3) + df["tract"].str.zfill(6)
    df["pop_total"] = pd.to_numeric(df["B02001_001E"], errors="coerce").clip(lower=0)
    df["pop_black"] = pd.to_numeric(df["B02001_003E"], errors="coerce").clip(lower=0)
    df["med_income"]= pd.to_numeric(df["B19013_001E"], errors="coerce")
    df["pov_denom"] = pd.to_numeric(df["B17001_001E"], errors="coerce").clip(lower=0)
    df["pov_below"] = pd.to_numeric(df["B17001_002E"], errors="coerce").clip(lower=0)
    df["clf"]       = pd.to_numeric(df["B23025_003E"], errors="coerce").clip(lower=0)
    df["unemployed"]= pd.to_numeric(df["B23025_005E"], errors="coerce").clip(lower=0)

    df["pct_black"]  = np.where(df["pop_total"] > 0,
                                (df["pop_black"] / df["pop_total"] * 100).clip(0, 100), np.nan)
    df["poverty_rate"]= np.where(df["pov_denom"] > 0,
                                 (df["pov_below"] / df["pov_denom"]).clip(0, 1), np.nan)
    df["unemp_rate"]  = np.where(df["clf"] > 0,
                                 (df["unemployed"] / df["clf"]).clip(0, 1), np.nan)

    print(f"  {len(df)} tracts fetched. Black % range: "
          f"{df['pct_black'].min():.1f}%-{df['pct_black'].max():.1f}%")
    return df[["geoid", "pop_total", "pop_black", "pct_black",
               "med_income", "poverty_rate", "unemp_rate"]]


# ─────────────────────────────────────────────────────────────────────────────
# Spatially-constrained Ward clustering (mirrors final_model2.ipynb Cell 8C)
# ─────────────────────────────────────────────────────────────────────────────

def build_shelby_communities(shelby_gdf: gpd.GeoDataFrame,
                             acs: pd.DataFrame,
                             n_communities: int = 6) -> gpd.GeoDataFrame:
    """
    Partition all Shelby tracts into N spatially contiguous communities using
    Ward agglomerative clustering with an adjacency connectivity constraint.

    Mirrors the notebook's Cell 8C methodology:
      - Features: economic indicators (income, poverty, unemployment)
        as proxies for IGS Economy pillar, signs flipped so higher = more vulnerable
      - Connectivity: tract adjacency matrix built from shapefile touches
      - Linkage: ward, metric: euclidean
    """
```

```
    print(f"  Building spatially-constrained Ward clustering ({n_communities} communities)...")

    df = shelby_gdf[["geoid", "geometry"]].merge(acs, on="geoid", how="left").copy()

    # Feature matrix — flip sign so all features encode vulnerability (higher = worse)
    features = pd.DataFrame({
        "neg_income":   -pd.to_numeric(df["med_income"],    errors="coerce"),
        "poverty_rate":  pd.to_numeric(df["poverty_rate"],   errors="coerce"),
        "unemp_rate":    pd.to_numeric(df["unemp_rate"],     errors="coerce"),
    }, index=df.index)

    X = SimpleImputer(strategy="median").fit_transform(features)
    X = StandardScaler().fit_transform(X)

    # Adjacency matrix for spatial connectivity constraint
    geoid_list = df["geoid"].tolist()
    idx_map    = {g: i for i, g in enumerate(geoid_list)}

    left  = shelby_gdf[["geoid", "geometry"]].rename(columns={"geoid": "g1"})
    right = shelby_gdf[["geoid", "geometry"]].rename(columns={"geoid": "g2"})
    adj   = gpd.sjoin(left, right, how="inner", predicate="touches")
    adj   = adj[adj["g1"] != adj["g2"]][["g1", "g2"]].drop_duplicates()

    valid = adj["g1"].isin(idx_map) & adj["g2"].isin(idx_map)
    adj   = adj[valid]
    r_idx = adj["g1"].map(idx_map).to_numpy()
    c_idx = adj["g2"].map(idx_map).to_numpy()

    connectivity = coo_matrix(
        (np.ones(len(r_idx), dtype=int), (r_idx, c_idx)),
        shape=(len(geoid_list), len(geoid_list)),
    )
    connectivity = connectivity.maximum(connectivity.T)

    # Ward clustering with spatial constraint
    model  = AgglomerativeClustering(
        n_clusters=n_communities,
        linkage="ward",
        connectivity=connectivity,
    )
    labels = model.fit_predict(X)

    df["shelby_community_id"] = [f"Shelby County community {int(x) + 1}" for x in labels]

    counts = df["shelby_community_id"].value_counts()
    print("  Community tract counts:")
    for name, cnt in counts.items():
        print(f"    {name}: {cnt} tracts")

    return df[["geoid", "geometry", "shelby_community_id"]].copy()


def select_high_growth_community(community_gdf: gpd.GeoDataFrame,
                                 acs: pd.DataFrame,
                                 low_growth_geoids: set) -> set:
    """
    Pick the best non-overlapping community as the high-growth comparator.
    Criteria (mirrors Cell 8F/8G): highest recovery economics (income, low poverty),
    no more than 10% overlap with the low-growth selected community.
```

```
    """
    best_score  = -np.inf
    best_geoids = set()

    for community_id, grp in community_gdf.groupby("shelby_community_id"):
        geoids  = set(grp["geoid"].dropna())
        overlap = len(geoids & low_growth_geoids) / max(len(geoids), 1)
        if overlap > 0.10:
            continue   # skip communities that overlap with low-growth area

        sub = acs[acs["geoid"].isin(geoids)]
        income  = sub["med_income"].median()
        poverty = sub["poverty_rate"].median()

        # Composite: high income + low poverty (same direction as IGS Economy pillar)
        score = income / 1000 - poverty * 100
        if score > best_score:
            best_score  = score
            best_geoids = geoids
            best_id     = community_id

    print(f"  High-growth comparator selected: {best_id} ({len(best_geoids)} tracts)")
    return best_geoids


# ─────────────────────────────────────────────────────────────────────────────
# Top-level data loader
# ─────────────────────────────────────────────────────────────────────────────

def load_and_classify() -> gpd.GeoDataFrame:
    print("Loading Shelby County geometry...")
    tn = gpd.read_file(SHP_PATH)
    tn["geoid"] = (tn["STATEFP"].str.zfill(2)
                   + tn["COUNTYFP"].str.zfill(3)
                   + tn["TRACTCE"].str.zfill(6))
    shelby = tn[(tn["STATEFP"] == "47") & (tn["COUNTYFP"] == "157")].copy()
    shelby = shelby.to_crs(3857)
    print(f"  {len(shelby)} Shelby County tracts")

    lg_gdf = gpd.read_file(LOW_GRW_PATH).copy()
    lg_gdf["geoid"]     = lg_gdf["geoid"].astype(str).str.zfill(11)
    low_growth_geoids   = set(lg_gdf["geoid"].dropna())
    print(f"  Low-growth tracts: {len(low_growth_geoids)}")

    acs = fetch_acs(2022)

    # Use cached cluster map if available, else recompute
    if CLUSTER_CACHE.exists():
        print(f"  Loading cached community map: {CLUSTER_CACHE}")
        community_gdf = gpd.read_file(CLUSTER_CACHE).copy()
        community_gdf["geoid"] = community_gdf["geoid"].astype(str).str.zfill(11)
        # Reproject to match shelby CRS
        community_gdf = community_gdf.set_crs(4326, allow_override=True).to_crs(3857)
    else:
        community_gdf = build_shelby_communities(shelby, acs, n_communities=6)
        # Save for reuse
        save_gdf = community_gdf.copy().to_crs(4326)
        save_gdf.to_file(CLUSTER_CACHE, driver="GeoJSON")
        print(f"  Saved community map: {CLUSTER_CACHE}")
```

```
    high_growth_geoids = select_high_growth_community(community_gdf, acs, low_growth_geoids)

    def assign_group(g):
        if g in low_growth_geoids:  return "low_growth"
        if g in high_growth_geoids: return "high_growth"
        return "rest"

    shelby["group"] = shelby["geoid"].map(assign_group)
    shelby = shelby.merge(
        acs[["geoid", "pct_black", "pop_black", "pop_total"]],
        on="geoid", how="left",
    )
    return shelby


# ─────────────────────────────────────────────────────────────────────────────
# Map
# ─────────────────────────────────────────────────────────────────────────────

def plot_black_population_choropleth(gdf: gpd.GeoDataFrame, output_path=None):
    """
    Choropleth of % Black by tract with community group outlines.
    Returns fig, ax. Saves to output_path if provided.
    """
    # Per-group summary stats
    stats = {}
    for grp in ["low_growth", "high_growth", "rest"]:
        sub   = gdf[gdf["group"] == grp]
        pop_b = pd.to_numeric(sub["pop_black"], errors="coerce").fillna(0)
        pct   = pd.to_numeric(sub["pct_black"],  errors="coerce")
        stats[grp] = {
            "n":           len(sub),
            "total_black": int(pop_b.sum()),
            "avg_pct":     float(pct.mean()),
        }

    fig, ax = plt.subplots(figsize=(15, 13))
    fig.patch.set_facecolor("white")
    ax.set_facecolor("white")

    # Choropleth: % Black, Blues scale
    norm = Normalize(vmin=0, vmax=100)
    gdf.plot(
        ax=ax, column="pct_black", cmap="Blues", norm=norm,
        edgecolor="#CCCCCC", linewidth=0.35,
        missing_kwds={"color": "#EEEEEE", "edgecolor": "#CCCCCC", "linewidth": 0.35},
        zorder=2,
    )

    # Community outlines only — bold, no fill
    OUTLINE = {
        "low_growth":  {"color": "#E8500A", "linewidth": 5.0},
        "high_growth": {"color": "#2CA02C", "linewidth": 5.0},
    }
    for grp, style in OUTLINE.items():
        dissolved = gdf[gdf["group"] == grp].dissolve()
        if dissolved.empty:
            continue
        dissolved.boundary.plot(
```

```
        ax=ax, color=style["color"], linewidth=style["linewidth"],
        capstyle="round", joinstyle="round", zorder=5,
    )

# Colorbar
sm = ScalarMappable(cmap="Blues", norm=norm)
sm.set_array([])
cbar = fig.colorbar(sm, ax=ax, fraction=0.026, pad=0.012, shrink=0.72, aspect=22)
cbar.set_label("% Black residents", fontsize=13, labelpad=10)
cbar.set_ticks([0, 20, 40, 60, 80, 100])
cbar.set_ticklabels(["0%", "20%", "40%", "60%", "80%", "100%"])
cbar.ax.tick_params(labelsize=11)

# Legend
ax.legend(
    handles=[
        mlines.Line2D([], [], color="#E8500A", linewidth=3.5,
                      label="Selected low-growth community outline"),
        mlines.Line2D([], [], color="#2CA02C", linewidth=3.5,
                      label="High-growth comparator outline"),
    ],
    title="Community outlines", title_fontsize=12,
    loc="upper left", fontsize=11,
    frameon=True, framealpha=0.95, edgecolor="#AAAAAA", handlelength=2.2,
)

# Title + subtitle
ax.set_title(
    "Shelby County: selected low-growth community and Black population share by tract",
    fontsize=18, fontweight="bold", pad=16, loc="left",
)
fig.text(
    ax.get_position().x0, 0.905,
    "Tract shading shows percent Black population using ACS tract averages for 2022-2024.",
    fontsize=12, color="#555555", fontstyle="italic",
    transform=fig.transFigure,
)

# Summary stats + story note
lg, hg, rs = stats["low_growth"], stats["high_growth"], stats["rest"]
summary = "\n".join([
    "Community snapshot (ACS tract averages, 2022-2024)",
    f"Selected low-growth community: {lg['avg_pct']:.0f}% Black | "
    f"{lg['total_black']:,} est. Black residents | {lg['n']} tracts",
    f"Rest of Shelby County: {rs['avg_pct']:.0f}% Black | "
    f"{rs['total_black']:,} est. Black residents | {rs['n']} tracts",
    f"High-growth comparator: {hg['avg_pct']:.0f}% Black | "
    f"{hg['total_black']:,} est. Black residents | {hg['n']} tracts",
    "",
    "Story note",
    (f"The selected low-growth community overlaps with the darker high-Black-share tract "
     f"corridor: {lg['avg_pct']:.0f}% Black versus {rs['avg_pct']:.0f}% in the rest of "
     f"Shelby County and {hg['avg_pct']:.0f}% in the high-growth comparator."),
])
fig.text(
    0.03, 0.01, summary,
    fontsize=10, color="#333333", va="bottom",
    bbox=dict(boxstyle="round,pad=0.7", facecolor="#F7F7F7",
              edgecolor="#BBBBBB", linewidth=0.9),
```

```
    )

    ax.set_axis_off()
    plt.subplots_adjust(bottom=0.20, top=0.91, left=0.01, right=0.87)

    if output_path:
        Path(output_path).parent.mkdir(parents=True, exist_ok=True)
        fig.savefig(output_path, dpi=200, bbox_inches="tight", facecolor="white")
        print(f"Saved: {output_path}")

    return fig, ax


# ── Entry point ──────────────────────────────────────────────────────────
if __name__ == "__main__":
    gdf = load_and_classify()
    print(f"\nGroup counts:\n{gdf['group'].value_counts().to_string()}")
    print(f"Missing pct_black: {gdf['pct_black'].isna().sum()}")
    print("\nRendering map...")
    fig, ax = plot_black_population_choropleth(gdf, output_path=OUT_PATH)
    plt.show()
```